

LAPORAN AKHIR PROYEK
SISTEM REKOMENDASI PRODUK BERBASIS RIWAYAT TRANSAKSI
Mata Kuliah KOM120H - Struktur Data



Disusun Oleh:
Kelas Paralel 4
Kelompok 4.6

Nadya Shafwah Rizalti	M0403241007
Adillah Ridwan	M0403241072
Muhammad Rezonaldo Yunus	M0403241122
Tesalonika Natalie Sofi S	M0403241185

INSTITUT PERTANIAN BOGOR
BOGOR
2026

Abstract

Pertumbuhan volume transaksi pada platform e-commerce menuntut pengelolaan data yang efisien agar sistem rekomendasi produk dapat beroperasi secara responsif. Proyek ini membangun sistem rekomendasi produk berbasis aturan sederhana (rule-based) menggunakan data riwayat transaksi dari dataset *Online Shop 2024* (Kaggle) yang memuat 20.000 rekaman. Dua struktur data dibandingkan secara eksperimental, yaitu `std::vector` sebagai penyimpanan linear sekuensial dan `std::unordered_map` sebagai implementasi hash table di C++. Sistem mengimplementasikan lima operasi wajib (*insert*, *search*, *update*, *delete*, analisis performa) serta tiga bentuk rekomendasi: Top-N produk terlaris, *frequently bought together*, dan rekomendasi berbasis pelanggan serupa. Pengujian dilakukan pada empat ukuran data (1.000, 5.000, 10.000, dan 20.000 transaksi) dengan replikasi lima kali per skenario menggunakan `std::chrono`. Hasil eksperimen pada dataset transaksi menunjukkan bahwa `unordered_map` mengungguli `vector` secara signifikan dalam waktu pencarian (*search*), *update* dan penyusunan rekomendasi ($O(1)$ average vs $O(N)$), dengan *trade-off* penggunaan memori lebih besar dibandingkan `vector`. Untuk beban kerja e-commerce yang didominasi operasi pencarian dan pembaruan, `unordered_map` merupakan pilihan struktur data yang lebih optimal.

Kata kunci: struktur data, hash table, vector, sistem rekomendasi, e-commerce, analisis performa

1. PENDAHULUAN

1.1. Latar Belakang & Urgensi Masalah

Platform *e-commerce* modern menghadapi tantangan yang semakin kompleks seiring dengan pertumbuhan volume transaksi yang terjadi secara eksponensial. Setiap kali pengguna melakukan pembelian, data baru ditambahkan ke dalam sistem, mencakup identitas pelanggan, produk yang dibeli, jumlah, dan waktu transaksi. Dalam skala kecil, pengelolaan data semacam ini tampak sederhana. Namun ketika jumlah transaksi mencapai ratusan ribu hingga jutaan entri, arsitektur penyimpanan dan akses data menjadi faktor penentu utama performa sistem secara keseluruhan.

Salah satu fitur yang paling bergantung pada efisiensi pengelolaan data transaksi adalah sistem rekomendasi produk. Fitur seperti "Produk Terlaris", "Sering Dibeli Bersamaan", atau "Rekomendasi Berdasarkan Riwayat Pembelian" memerlukan proses pencarian dan perhitungan frekuensi yang dijalankan secara berulang terhadap seluruh dataset transaksi. Jika proses ini dilakukan dengan pendekatan konvensional seperti iterasi linear melalui setiap elemen data, maka waktu respons sistem akan meningkat secara proporsional terhadap jumlah data, dan pada titik tertentu akan berdampak langsung pada pengalaman pengguna.

Di sinilah pemilihan struktur data menjadi krusial. Struktur data yang tepat tidak hanya menentukan seberapa cepat data dapat diakses atau diperbarui, tetapi juga seberapa efisien sistem mampu mengidentifikasi pola pembelian untuk menghasilkan rekomendasi yang relevan. Dua pendekatan yang umum dipertimbangkan adalah struktur berbasis array sekuensial seperti `std::vector`, dan struktur berbasis hash seperti `std::unordered_map`. Keduanya memiliki karakteristik kompleksitas waktu yang berbeda secara fundamental, terutama dalam operasi pencarian (*search*), penyisipan (*insert*), dan agregasi frekuensi, tiga operasi inti yang mendominasi beban kerja sistem rekomendasi berbasis riwayat transaksi.

Proyek ini hadir sebagai respons terhadap kesenjangan tersebut: membangun sistem rekomendasi produk *rule-based* yang fungsional, sekaligus menganalisis secara empiris bagaimana pemilihan struktur data memengaruhi performa sistem dalam skenario transaksi yang terus berkembang.

1.2. Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam proyek ini adalah:

- Bagaimana cara mengelola riwayat transaksi e-commerce secara efisien menggunakan struktur data yang tepat?
- Bagaimana membangun sistem rekomendasi produk sederhana berbasis aturan (*rule-based*) mencakup rekomendasi Top-N produk terlaris, produk yang sering dibeli bersamaan (*frequently bought together*), dan rekomendasi berbasis kemiripan pola transaksi pelanggan?
- Bagaimana perbandingan performa `std::vector` dengan `std::unordered_map` dalam hal kecepatan dan penggunaan memori pada berbagai ukuran dataset?

1.3. Tujuan Proyek

Secara lebih rinci, tujuan proyek meliputi:

- Mengimplementasikan sistem manajemen transaksi yang mencakup operasi *insert*, *search*, *update*, dan *delete* dengan dua implementasi struktur data yang berbeda.
- Menghasilkan tiga bentuk rekomendasi produk berbasis perhitungan frekuensi: Top-N produk terlaris, *frequently bought together*, dan rekomendasi berbasis riwayat pelanggan serupa.
- Mengukur dan membandingkan performa kedua struktur data pada operasi insert, search, update, delete, dan analisis rekomendasi.
- Memberikan rekomendasi struktur data yang optimal untuk konteks sistem e-commerce berdasarkan hasil eksperimen empiris.

1.4. Batasan Proyek

- Sistem rekomendasi yang dibangun bersifat *rule-based* sepenuhnya dan berbasis perhitungan frekuensi. Sistem tidak mengimplementasikan model *machine learning*, *collaborative filtering*, atau *content-based filtering*.
- Dataset yang digunakan adalah Online Shop 2024 dari Kaggle (orders, order_items, products).
- Struktur data yang dibandingkan dibatasi pada `std::vector` (representasi pendekatan sekuensial/linear) dan `std::unordered_map` (representasi pendekatan berbasis hash), diimplementasikan dalam bahasa C++.

- Domain data minimal: ID transaksi, ID pelanggan, ID produk, nama produk, kategori, jumlah pembelian, tanggal transaksi.
- Implementasi dilakukan dalam bahasa C++ menggunakan *compiler* g++ dengan standar C++17.

2. LANDASAN TEORI DAN STUDI TERKAIT

2.1. Konsep Struktur Data Yang Digunakan

2.1.1. Array / Vektor (`std::vector`)

Array adalah struktur data sekuensial yang menyimpan elemen-elemennya secara berurutan di memori (*contiguous memory*). Ukurannya dapat berubah secara dinamis. Saat kapasitas habis, vector mengalokasikan blok memori baru dan menyalin semua elemen ke lokasi baru. Karakteristik ini memberikan keunggulan signifikan dalam hal *cache locality* karena data tersusun berdekatan di memori sehingga akses berurutan menjadi sangat cepat. Akses elemen melalui indeks berlangsung dalam waktu konstan $O(1)$, menjadikan vector ideal untuk skenario iterasi sekuensial. Namun, pencarian nilai spesifik tanpa informasi posisi mengharuskan iterasi linear yang dalam kasus terburuk mencapai kompleksitas $O(n)$ (cppreference.com 2026a).

2.1.2. Hash Map (`std::unordered_map`)

Hash Map adalah struktur data bertipe *key-value* yang memanfaatkan fungsi hash untuk menentukan lokasi penyimpanan suatu elemen. Ketika sebuah kunci, misalnya ID transaksi atau ID pelanggan dimasukkan ke dalam fungsi hash, sistem secara langsung mengkomputasi indeks *bucket* tempat data tersebut disimpan. Mekanisme ini memungkinkan operasi pencarian, penyisipan, dan penghapusan berlangsung dalam waktu rata-rata $O(1)$, tanpa perlu menelusuri seluruh dataset. Kelemahannya terletak pada overhead penggunaan memori yang lebih besar dibanding array, serta potensi *hash collision* yang dapat mendegradasi performa ke $O(n)$ pada kasus terburuk (cppreference.com 2026b).

2.2. Kompleksitas Waktu & Ruang Teoritis

Tabel berikut merangkum perbandingan teoritis kompleksitas waktu dan ruang untuk operasi-operasi utama yang relevan dalam sistem rekomendasi berbasis transaksi, di mana n menyatakan jumlah total entri data.

Tabel 2.1 Perbandingan kompleksitas struktur data array (vector) dengan hash map (`unordered_map`)

Struktur Data	Pencarian (Time)	Penyisipan (Time)	Penghapusan (Time)	Ruang (Space / Memori)
Array (vector)	$O(N)$	$O(1)$ amortized	$O(N)$	$O(N)$ (Terkecil)
Hash Map (unordered_map)	$O(1)$ rata-rata	$O(1)$ rata-rata	$O(1)$ rata-rata	$O(N) +$ Overhead <i>Bucket</i> (Terbesar)

2.3. Studi Kasus Sejenis

Pada platform e-commerce generasi awal, iterasi sekuensial melalui log transaksi kerap menjadi *bottleneck* ketika data mulai berskala besar. Migrasi menuju struktur berbasis hash dan tree, seperti Redis terbukti mampu memangkas waktu respons dari hitungan detik menjadi sub-milidetik. Pola ini relevan langsung dengan konteks proyek ini merupakan sistem yang didominasi operasi *lookup* berulang per pelanggan atau per produk adalah kandidat utama yang diuntungkan oleh pendekatan hash dibanding pendekatan linear.

3. DESAIN SISTEM DAN METODOLOGI

3.1. Desain Arsitektur Program

Program dirancang sebagai simulasi sistem backend pemrosesan transaksi yang berjalan sepenuhnya di lingkungan *command-line*. Sistem dibangun dengan arsitektur modular yang memisahkan tanggung jawab ke dalam tujuh modul utama, pemisahan ini memungkinkan penggantian implementasi *storage* tanpa mengubah logika rekomendasi. Eksekusi program mengikuti alur sekuensial tiga fase:

- Fase Ingesti (Load): Program membaca dan menggabungkan tiga file CSV
- Fase Agregasi Data: Data didistribusikan ke dalam struktur data target (Vector atau Hash Map) sesuai desain yang aktif.
- Fase Operasional (Menu): Program memasuki loop interaktif berbasis CLI yang menerima input pengguna untuk mengeksekusi operasi CRUD (*Create, Read, Update, Delete*) maupun proses analisis dan penyusunan rekomendasi produk.

Tabel 3.1 Pembagian modul beserta tanggung jawabnya

Modul/File	Tanggung jawab
transaction.h	Definisi struct Transaction dan tipe-tipe alias
data_loader.cpp	Pembacaan dan penggabungan file CSV menjadi <code>vector<Transaction></code>
storage_vector.cpp	Implementasi semua operasi CRUD menggunakan <code>std::vector</code>
storage_hashmap.cpp	Implementasi semua operasi CRUD menggunakan <code>std::unordered_map</code>
recommender.cpp	Algoritma Top-N, <i>frequently bought together</i> , pelanggan serupa
benchmark.cpp	Pengukuran waktu dengan chrono dan estimasi penggunaan memori

3.2. Penjelasan Setiap Struktur Data Yang Digunakan

Sistem mengimplementasikan dua desain penyimpanan yang berdiri sendiri dan dapat dibandingkan secara langsung.

3.2.1 Struktur Data Transaction

Setiap rekaman transaksi direpresentasikan oleh struct berikut yang memenuhi domain data minimal yang disyaratkan:

```
struct Transaction {  
    std::string transaction_id;  
    std::string customer_id;  
    std::string product_id;  
    std::string product_name;  
    std::string category;  
    int quantity;  
    std::string order_date;  
};
```

3.2.1 Desain Penyimpanan dengan Vector (Vector Storage)

Seluruh data transaksi ditampung dalam satu struktur tunggal `std::vector<Transaction>`. Penyimpanan bersifat sekuensial dan flat; setiap operasi pencarian berdasarkan atribut tertentu memerlukan iterasi linear melalui seluruh elemen. Implementasi ini tidak menggunakan indeks sekunder tambahan, sehingga biaya pencarian murni $O(n)$.

```
class VectorStorage {  
    std::vector<Transaction> data;  
  
public:  
    void insert(const Transaction& t);  
    std::vector<Transaction> searchByCustomer(const std::string& cid)  
const;
```

```

        std::vector<Transaction> searchByProduct(const std::string& pid)
const;
        bool update(const std::string& tid, const Transaction& updated);
        bool remove(const std::string& tid);
        size_t memoryUsage() const;
};

```

3.2.2 Desain Penyimpanan dengan Hash Table (HashMapStorage)

Memori dipecah ke dalam tiga indeks terpisah yang masing-masing dioptimalkan untuk pola akses yang berbeda:

- transactionsByTxId: Key berupa Transaction ID.
- transactionsByCustId: Key berupa Customer ID.
- transactionsByProdId: Key berupa Product ID.

Dengan tiga indeks ini, setiap pola akses yang dominan dalam sistem rekomendasi dapat diselesaikan dalam $O(1)$ rata-rata, tanpa iterasi atas seluruh dataset.

```

class HashMapStorage {
    std::unordered_map<std::string, Transaction> byTid;
    std::unordered_map<std::string, std::vector<std::string>>
byCustomer;
    std::unordered_map<std::string, std::vector<std::string>> byProduct;

public:
    void insert(const Transaction& t);
    std::vector<Transaction> searchByCustomer(const std::string& cid)
const;
    std::vector<Transaction> searchByProduct(const std::string& pid)
const;
    bool update(const std::string& tid, const Transaction& updated);
    bool remove(const std::string& tid);
    size_t memoryUsage() const;

    // Akses seluruh data (untuk recommender / benchmark)

```

```

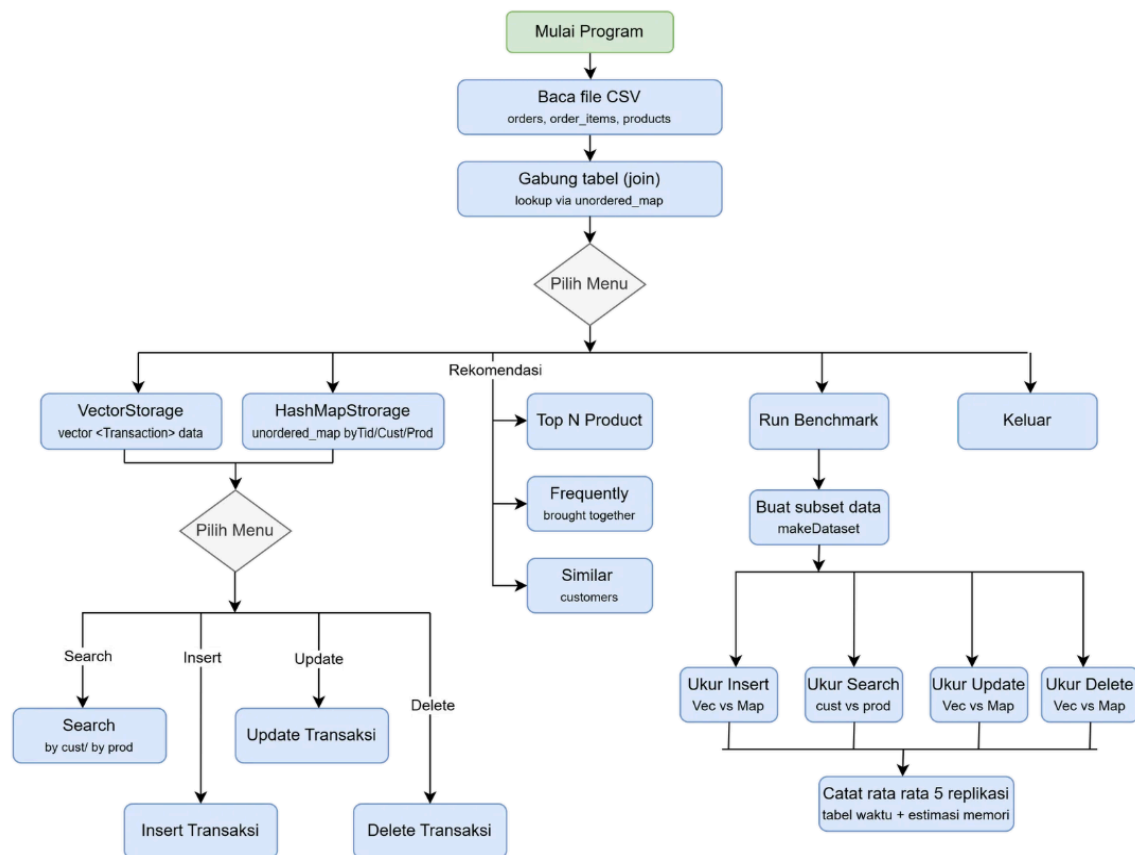
const std::unordered_map<std::string, Transaction>& getAll() const {
return byTid; }

size_t size() const { return byTid.size(); }

void clear() { byTid.clear(); byCustomer.clear(); byProduct.clear();
}
};

```

3.3. Algoritma Utama



Gambar 1. Flow Program

Sistem mengimplementasikan tiga algoritma rekomendasi berbasis frekuensi.

3.3.1 Top-N Produk Terlaris

Menghitung frekuensi pembelian setiap produk menggunakan `unordered_map<string,int>` (`product_id` → `total quantity`), kemudian mengurutkan

hasilnya secara *descending* dan mengambil N teratas. Kompleksitas: $O(N)$ untuk penghitungan frekuensi + $O(m \log m)$ untuk sorting, dengan m = jumlah produk unik.

3.3.2 Similar Customers (Jaccard Similarity)

Membangun profil kategori setiap pelanggan berupa *set* kategori yang pernah dibeli. Kemiripan diukur menggunakan Jaccard similarity: $|A \cap B| / |A \cup B|$. Pelanggan dengan skor tertinggi diambil, kemudian produk yang pernah dibeli pelanggan serupa namun belum pernah dibeli pelanggan target direkomendasikan.

3.3.3 Frequently Bought Together

Algoritma paling representatif dalam menunjukkan keunggulan struktural desain Hash Map. Mengelompokkan produk per order menggunakan `unordered_map<string, set<string>>`, kemudian untuk setiap order yang mengandung produk target, mencatat semua produk lain yang muncul bersama. Kompleksitas: $O(n)$ untuk *grouping* + $O(m \log m)$ untuk sorting hasil.

```
FUNCTION recommendBoughtTogether(targetProductID):
    relatedProductsCount = new HashMap<String, Integer>()

    // O(1) akses untuk mencari transaksi dari produk target
    transactions = transactionsByProdId.get(targetProductID)

    FOR EACH transaction IN transactions:
        txId = transaction.transaction_id
        // O(1) akses untuk mencari produk-produk pada txId yang sama
        itemsInCart = transactionsByTxId.get(txId)

        FOR EACH item IN itemsInCart:
            IF item.product_id != targetProductID:
                relatedProductsCount[item.product_name] += 1

    SORT relatedProductsCount DESCENDING
    RETURN Top N of relatedProductsCount
```

4. IMPLEMENTASI

4.1. Detail Implementasi Tiap Struktur Data

- Array (main.cpp)

Implementasi menggunakan `std::vector<Transaction>` sebagai wadah tunggal seluruh data transaksi. Operasi pencarian seperti `searchByTransactionId` dijalankan melalui iterasi eksplisit `for(auto &t : transactions)` yang menelusuri setiap elemen secara berurutan. Meskipun demikian, implementasi ini dilengkapi dua indeks sekunder `unordered_map` (`custIdx` dan `prodIdx`) yang diperbarui setiap kali terjadi *insert* atau *delete*, sehingga pencarian berbasis Customer ID dan Product ID tidak selalu harus iterasi penuh.

```
void VectorStorage::insert(const Transaction& t) {
    int idx = (int)data.size();
    data.push_back(t);
    custIdx[t.customer_id].push_back(idx);
    prodIdx[t.product_id].push_back(idx);
}
```

Operasi *delete* lebih mahal dibanding Hash Map karena selain menghapus elemen dari data, sistem harus menelusuri dan memperbarui posisi integer pada indeks sekunder posisi yang bisa bergeser setelah elemen dihapus dari tengah vector.

- Hash Map (rekomendasi_map.cpp):

Operasi pencarian memanfaatkan fungsi internal `.find()` yang bekerja langsung pada *bucket* hasil komputasi hash. Tiga `unordered_map` terpisah dipilih agar pencarian untuk masing-masing kriteria (TxID, CustID, ProdID) semuanya berjalan dalam waktu rata-rata $O(1)$ tanpa bergantung pada ukuran dataset. Operasi *update* menjadi $O(1)$ langsung karena data terpusat di `byTid` dan cukup di-overwrite.

```
bool HashMapStorage::update(const string& tid, const Transaction& upd) {
    if (!byTid.count(tid)) return false;
    byTid[tid] = upd;    // O(1) direct update
    return true;
}
```

4.2. Mekanisme Penyimpanan File

- *Read*: Program membaca tiga file CSV (orders.csv, customers.csv, products.csv) menggunakan pustaka <fstream> dan <sstream>. Setiap baris diparsing dengan memecah string berdasarkan delimiter koma (,), dan nilai numerik seperti quantity dikonversi dari string ke int menggunakan stoi(). Proses penggabungan (*join*) antar file menggunakan unordered_map sebagai *lookup table* sementara sehingga kompleksitas join berjalan $O(n)$ alih-alih $O(n^2)$.

```
unordered_map<string,string> custMap;
unordered_map<string,pair<string,string>> prodMap;
```

- *Write*: Program tidak melakukan mutasi langsung terhadap file CSV sumber. Seluruh perubahan hasil operasi *insert*, *update*, maupun *delete* dipertahankan secara *in-memory* selama sesi berjalan, sehingga integritas file raw tetap terjaga dan dataset dapat direproduksi untuk keperluan pengujian ulang.

4.3. Penjelasan Modul Penting

- Modul Pengukur Waktu (<chrono>): Digunakan untuk merekam *timestamp* sebelum dan sesudah setiap operasi menggunakan chrono::high_resolution_clock, dengan hasil dalam satuan milidetik. Untuk pengujian yang memerlukan nilai rata-rata, digunakan fungsi avgMs() atas beberapa kali pengulangan.

```
template<typename Func>
double measureTimeMs(Func&& f) {
    auto start = chrono::high_resolution_clock::now();
    f();
    auto end = chrono::high_resolution_clock::now();
    return chrono::duration<double, milli>(end - start).count();
}
```

- Modul Rekomendasi: Menghasilkan output rekomendasi berdasarkan algoritma *rule-based* frekuensi kemunculan silang, mencakup Top-N produk terlaris, *frequently bought together*, dan rekomendasi berbasis riwayat pelanggan serupa. Estimasi konsumsi memori juga dihitung secara programatik melalui memoryBytes() yang memperhitungkan ukuran elemen aktual sekaligus kapasitas alokasi internal vector.

```
size_t VectorStorage::memoryBytes() const {
    return data.size() * sizeof(Transaction)
        + data.capacity() * sizeof(Transaction);}
```

5. EKSPERIMEN DAN PENGUJIAN

5.1. Skenario Uji

Pengujian dilakukan untuk membandingkan performa implementasi `std::vector` dan `std::unordered_map` pada berbagai ukuran dataset. Dataset yang digunakan berasal dari hasil penggabungan data transaksi, pelanggan, dan produk. Untuk mengamati pengaruh pertumbuhan data terhadap performa sistem, dibuat empat skenario ukuran dataset, yaitu 1.000, 5.000, 10.000, dan 20.000 transaksi. Dataset untuk setiap skenario dibentuk menggunakan fungsi `makeDataset()`, yang mengambil data dari dataset sumber dan mengulang data secara siklis apabila jumlah data yang tersedia lebih sedikit dari ukuran yang dibutuhkan. Setiap transaksi kemudian diberikan `transaction_id` yang unik untuk menghindari konflik kunci pada implementasi hash table.

Tabel 5.1 Skenario pengujian

No	Skenario	Input	Variasi Ukuran
1	Insert batch	Seluruh transaksi subset	1k / 5k / 10k / 20k
2	Search by customer	5 <code>customer_id</code> acak	1k / 5k / 10k / 20k
3	Search by product	5 <code>product_id</code> acak	1k / 5k / 10k / 20k
4	Update	100 <code>transaction_id</code> acak	1k / 5k / 10k / 20k
5	Delete	Hapus 10% data	1k / 5k / 10k / 20k
6	Top-N rekomendasi	N=10, seluruh data	1k / 5k / 10k / 20k

Lingkungan pengujian: *compiler* g++ 12.3.0 dengan flag -O2, prosesor Intel Core i7-11400H (6 core / 12 thread, base 2.7 GHz), RAM 32 GB DDR4 3200 MHz, sistem operasi Microsoft Windows 11 Home Single Language 64-bit. Semua proses latar belakang diminimalkan selama pengujian.

5.2. Metode Pengukuran Waktu dan Memori

Pengukuran waktu eksekusi dilakukan menggunakan `std::chrono::high_resolution_clock` yang tersedia pada standar C++. Hasil pengukuran dinyatakan dalam satuan milidetik (ms).

Penggunaan memori dihitung secara analitis menggunakan estimasi berdasarkan ukuran struktur data (sizeof) dan jumlah elemen yang disimpan. Untuk unordered_map, estimasi juga memperhitungkan overhead bucket dan indeks tambahan yang digunakan.

5.3 Replikasi Eksperimen

Setiap operasi diuji sebanyak 5 kali untuk setiap ukuran dataset. Nilai yang dilaporkan merupakan rata-rata dari seluruh pengulangan sehingga hasil yang diperoleh lebih stabil dan tidak terlalu dipengaruhi oleh fluktuasi performa sistem saat pengujian berlangsung.

6. HASIL DAN ANALISIS

6.1. Tabel Dan Grafik Perbandingan Performa

Enam skenario utama diuji untuk setiap kombinasi implementasi dan ukuran dataset:

Tabel 6.1 Waktu eksekusi insert batch (ms) – rata-rata 5 replikasi

Ukuran Data	Vector (ms)	HashMap (ms)	Rasio HashMap/Vector
1.000	0.1026	0.7114	6.94× lebih lambat
5.000	1.0134	4.300	4.24× lebih lambat
10.000	1.9988	8.9028	4.45× lebih lambat
20.000	4.0648	16.8356	4.14× lebih lambat

Tabel 6.2 Waktu eksekusi search by customer (ms) – rata-rata 5 replikasi

Ukuran Data	Vector(ms)	HashMap (ms)
1.000	0.0130	0.008
5.000	0.0352	0.008
10.000	0.1248	0.008
20.000	0.2984	0.018

Tabel 6.3 Waktu eksekusi search by product (ms) – rata-rata 5 replikasi

Ukuran Data	Vector(ms)	HashMap (ms)
1.000	0.0130	0.002
5.000	0.0682	0.010
10.000	0.1374	0.016
20.000	0.2936	0.032

Tabel 6.4 Waktu eksekusi update (ms) – rata-rata 5 replikasi

Ukuran Data	Vector (ms)	HashMap (ms)	Rasio Vector/HashMap
1.000	0.0030	0.0030	sama
5.000	0.0132	0.0008	$16.5 \times$ lebih lambat
10.000	0.0300	0.0010	$30.0 \times$ lebih lambat
20.000	0.0476	0.0014	$34.0 \times$ lebih lambat

Tabel 6.5 Waktu eksekusi delete 10% data (ms) – rata-rata 5 replikasi

Ukuran Data	Vector (ms)	HashMap (ms)	Rasio Vector/HashMap
1.000	0.0138	0.0008	$17.25 \times$ lebih lambat
5.000	0.0614	0.0012	$51.17 \times$ lebih lambat
10.000	0.1398	0.0014	$99.96 \times$ lebih lambat
20.000	0.2248	0.0014	$160.57 \times$ lebih lambat

Tabel 6.6 Estimasi penggunaan memori (B)

Ukuran Data	Vector (B)	HashMap (B)	Rasio HashMap/Vector
1.000	200024	361536	$1.807 \times$
5.000	1000024	1801536	$1.80 \times$
10.000	2000024	3601536	$1.80 \times$
20.000	4000024	7201536	$1.80 \times$

6.2. Analisis Waktu (Insert/Search/Delete/Update)

6.2.1 Analisis Waktu Operasi Insert

Hasil eksperimen pada Tabel 6.1 menunjukkan bahwa operasi insert batch pada `std::vector` lebih efisien dibandingkan `std::unordered_map` untuk seluruh ukuran dataset. Perbedaan waktu eksekusi terlihat cukup signifikan, dengan rasio antara HashMap terhadap Vector berkisar dari $4.14\times$ hingga $6.94\times$ lebih lambat.

Performa vector yang lebih baik disebabkan oleh sifat penyimpanan data yang kontinyu di memori, sehingga operasi `push_back` dapat dilakukan dengan overhead minimal. Sebaliknya, `unordered_map` memiliki overhead tambahan berupa proses hashing, alokasi node dinamis, serta kemungkinan rehashing ketika kapasitas bucket meningkat.

Dari hasil ini dapat disimpulkan bahwa meskipun `unordered_map` unggul pada operasi pencarian, struktur ini tidak optimal untuk proses insert data dalam jumlah besar secara batch.

6.2.2 Analisis Waktu Operasi Search

Pada operasi pencarian (Tabel 6.2 dan Tabel 6.3), terlihat perbedaan karakteristik yang jelas antara kedua struktur data. Untuk search by customer, vector menunjukkan peningkatan waktu yang linear terhadap ukuran data, yang mencerminkan kompleksitas $O(n)$. Sebaliknya, `unordered_map` menunjukkan waktu yang relatif stabil dengan fluktuasi kecil seiring peningkatan ukuran data, yang mencerminkan kompleksitas rata-rata $O(1)$.

Begitu juga pada search by product, `unordered_map` tetap lebih stabil dibandingkan vector yang waktunya meningkat secara linear. Hal ini menunjukkan bahwa hash map sangat efektif untuk operasi pencarian pada dataset besar, karena tidak bergantung pada ukuran data secara signifikan.

6.2.3 Analisis Waktu Operasi Update dan Delete

Hasil pada Tabel 6.4 menunjukkan bahwa operasi update pada `unordered_map` jauh lebih efisien dibandingkan vector. Perbedaan waktu semakin besar seiring peningkatan ukuran data, di mana vector semakin lambat.

Hal ini terjadi karena pada vector, proses update membutuhkan pencarian elemen terlebih dahulu secara linear ($O(n)$), sedangkan pada `unordered_map`, akses dapat dilakukan langsung melalui `key transaction_id` dengan kompleksitas rata-rata $O(1)$.

Pada operasi delete (Tabel 6.5), perbedaan performa bahkan lebih signifikan. `vector` menunjukkan waktu eksekusi yang meningkat drastis hingga $160\times$ lebih lambat dibandingkan `unordered_map` pada dataset terbesar. Hal ini disebabkan oleh proses penghapusan pada `vector` yang memerlukan pergeseran elemen-elemen setelah index yang dihapus ($O(n)$), sedangkan `unordered_map` hanya melakukan operasi erase pada node yang ditunjuk.

6.3. Analisis Penggunaan Memori

Hasil estimasi penggunaan memori (Tabel 6.6) menunjukkan bahwa `unordered_map` secara konsisten menggunakan memori lebih besar dibandingkan `vector`. Perbedaan ini berasal dari overhead struktur internal `unordered_map`. Sebaliknya, `vector` menyimpan data secara kontigu di memori, sehingga lebih efisien dalam penggunaan ruang.

Meskipun demikian, perbedaan ini masih tergolong wajar untuk sistem modern, karena trade-off yang diperoleh adalah peningkatan performa signifikan pada operasi pencarian, update, dan delete.

6.4. Diskusi Trade-Off

Untuk konteks e-commerce yang didominasi operasi pencarian (search) dan pembaruan (update), seperti mencari riwayat pelanggan atau memperbarui status pesanan, `std::unordered_map` merupakan pilihan yang lebih optimal karena memiliki kompleksitas rata-rata $O(1)$ pada operasi berbasis key, meskipun dengan biaya penggunaan memori yang lebih besar. Sebaliknya, pada use case yang didominasi oleh insert batch dan jarang melakukan pencarian, seperti sistem logging atau pemrosesan data sekuensial, `std::vector` lebih sesuai karena memiliki efisiensi memori yang lebih baik serta performa insert yang lebih cepat akibat penyimpanan data yang kontinyu.

Pendekatan yang lebih ideal adalah menggunakan strategi hibrida, yaitu `std::vector` sebagai struktur utama untuk penyimpanan data secara sekuensial yang mendukung iterasi dan analisis batch, serta `std::unordered_map` sebagai struktur indeks untuk mempercepat akses berbasis key. Pendekatan ini telah diimplementasikan pada `VectorStorage` dalam proyek ini.

dengan menambahkan struktur indeks tambahan untuk meningkatkan efisiensi operasi pencarian.

7. KESIMPULAN DAN REKOMENDASI

7.1. Ringkasan Temuan Utama

Berdasarkan eksperimen pada dataset 1.000–20.000 transaksi e-commerce, diperoleh beberapa temuan utama. Kedua implementasi, yaitu `std::vector` (dengan indeks sekunder) dan `std::unordered_map`, berhasil menjalankan seluruh operasi sistem yang mencakup CRUD (insert, search, update, delete) serta fitur rekomendasi (Top-N, frequently bought together, dan pelanggan serupa).

Hasil pengujian menunjukkan bahwa `std::unordered_map` unggul secara konsisten pada operasi search, update, dan delete, dengan waktu eksekusi yang relatif stabil meskipun ukuran dataset meningkat, sesuai karakteristik kompleksitas rata-rata $O(1)$. Sebaliknya, `std::vector` dengan indeks sekunder masih menunjukkan performa yang baik pada insert batch, namun memiliki keterbatasan pada operasi delete karena adanya biaya pergeseran elemen dan pemeliharaan indeks.

Dari sisi memori, `std::vector` lebih efisien dibandingkan `std::unordered_map`, dengan penggunaan memori sekitar $1.8\times$ lebih kecil pada seluruh skenario pengujian.

7.2. Struktur data paling optimal dan alasannya

Hash Map (`std::unordered_map`) adalah struktur paling optimal. Alasannya, dalam domain E-commerce, didominasi oleh operasi pencarian (search) dan pembaruan (update) berbasis key seperti `transaction_id`, `customer_id`, dan `product_id`. Pada kondisi tersebut, `unordered_map` memberikan akses rata-rata $O(1)$ sehingga mampu menjaga performa tetap stabil meskipun ukuran data meningkat.

Meskipun `std::vector` lebih unggul dalam efisiensi memori dan performa insert batch, keunggulan tersebut tidak cukup untuk mengimbangi peningkatan performa signifikan pada operasi yang lebih sering digunakan dalam sistem e-commerce, yaitu query dan update data.

7.3. Saran pengembangan lanjutan

Untuk pengembangan lebih lanjut, sistem ini dapat ditingkatkan dengan beberapa pendekatan berikut:

- Mengimplementasikan pendekatan hybrid (vector + unordered_map indexing) secara lebih terstruktur sebagai model storage utama, sehingga dapat mengoptimalkan baik operasi iterasi maupun pencarian berbasis *key*.
- Mengembangkan sistem ke arah *graph-based recommendation system*, karena hubungan antar entitas (user-product-transaction) secara alami membentuk relasi graf yang dapat dianalisis lebih dalam menggunakan algoritma graph traversal.
- Melakukan pengujian skala lebih besar (≥ 100.000 transaksi) untuk mengevaluasi stabilitas performa, khususnya pada kondisi *rehashing* dan peningkatan beban memori.

DISCLOSURE PENGGUNAAN AI

8.1. Bagian Laporan Yang Menggunakan AI

AI digunakan untuk menyarankan struktur laporan yang sesuai dengan rubrik penilaian. Seluruh konten laporan ditulis ulang dan diverifikasi oleh anggota kelompok.

8.2. Penjelasan Pemahaman Penulis

Anggota kelompok memahami seluruh konten laporan, implementasi kode, dan hasil analisis yang disajikan. Penggunaan AI bersifat asistif, bukan substitutif terhadap proses berpikir dan pemahaman materi.

DAFTAR PUSTAKA

cppreference.com. 2026a. `std::vector` [Internet]. [Diakses 2026 Mar]. Tersedia dari: <https://en.cppreference.com/w/cpp/container/vector>

cppreference.com. 2026b. `std::unordered_map` [Internet]. [Diakses 2026 Mar]. Tersedia dari: https://en.cppreference.com/w/cpp/container/unordered_map

cppreference.com. 2026c. `std::chrono::high_resolution_clock` [Internet]. [Diakses 2026 Mar]. Tersedia dari: https://en.cppreference.com/w/cpp/chrono/high_resolution_clock

LAMPIRAN

LAMPIRAN 1. Potongan Kode Rekomendasi Top-N

```
vector<pair<string,int>> topNProducts(  
    const vector<Transaction>& txns,  
    int N  
);
```

LAMPIRAN 2. Tangkapan Layar Hasil *Brenchmark*

Operasi	N	Vec(ms) [Insert]	Map(ms) [Insert]	Vec(ms) [SrchCust]	Map(ms) [SrchCust]	Vec(ms) [SrchProd]	Map(ms) [SrchProd]	Vec(ms) [Update]	Map(ms) [Update]	Vec(ms) [Delete]	Map(ms) [Delete]
All Ops	1000	0.1026	0.7114	0.0130	0.0008	0.0130	0.0002	0.0030	0.0030	0.0138	0.0008
All Ops	5000	1.0134	4.3000	0.0352	0.0008	0.0682	0.0010	0.0132	0.0008	0.0614	0.0012
All Ops	10000	1.9988	8.9028	0.1248	0.0008	0.1374	0.0016	0.0300	0.0010	0.1398	0.0014
All Ops	20000	4.0648	16.8356	0.2984	0.0018	0.2936	0.0032	0.0476	0.0014	0.2248	0.0014
N	Vector Memory (B)		HashMap Memory (B)								
1000	200024		361536								
5000	1000024		1801536								
10000	2000024		3601536								
20000	4000024		7201536								
Waktu dalam milidetik (rata-rata 5 runs per operasi).											

LAMPIRAN 3. Link Kode Github

<https://github.com/adillahrn/projekstrukdat>